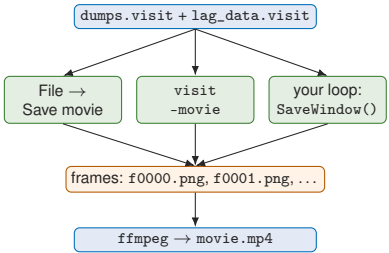
VISIT: MAKING MOVIES

Frames first, then one encoding command: the Save movie wizard, `visit -movie` with a session or a script, a loop of your own, and ffmpeg; fixed colour limits, a time stamp, the frame rate you asked for, and files that play everywhere

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part V • Chapter 30



pictures and numbers: VisIt 3.5.0 and ffmpeg 6.1.1 on IB/11 (the heart-tube racetrack); the wizard's windows: VisIt 2.5.2, from the 2014–2024 tutorials

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

- Prepare a plot for a movie: fixed colour limits, a fixed view, one legend, a time stamp, no refinement-level lines.
- Make the frames three ways: the **Save movie** wizard, `visit -movie` with a saved session or a script, and a loop of your own.
- Turn frames into an MP4 that plays in slides, browsers and phones with one ffmpeg command; make a GIF; put two runs side by side.
- Choose the frame rate and the dump interval for the movie you want.
- Recognise the silent failures: no frames written, a structure that stands still, a frame rate you did not ask for, a file that will not play.

> A movie is a folder of frames and one encoding command. Keep the frames, and fix everything that must not change from frame to frame (limits, view, legend) before the first one.

WHERE THIS CHAPTER COMES FROM

- N. Battista's tutorial, "Part III: Let's make groovy movies!" (2014, for UNC's Kure and Killdevil; the 2018 MATH 769 version is "Run your first IBAMR simulation"): the first walk-through from `lag_data.visit` to a movie: **File** → **Save Movie**, **New simple movie**, **MPEG Movie** and the right arrow, start, end, frame rate and stride, a folder and a title ("cleverly titled FIRST_SIMULATION_MOVIE"), an e-mail if you like, and "Now, use currently allocated processors".
- "VISIT – How to visualize IBAMR Data" (2020, slides 21–23 and 31; again in the 2021 deck) and the 2024 IB2d practical (slides 25–27): the same wizard steps with the 2014 pictures (the practical stops there, with **MPEG Movie**); then the 2020 and 2021 decks, like "Lecture 9: Sampling, averaging, and analyzing data in Visit" (2019, slide 26), say "Save as mpeg is not the best option": save PNG or JPEG images and "Put the set of images back together as a movie using VirtualDub (free for PC) or Quicktime Pro".
- L. Waldrop's guide "Data analysis in VisIt" (MATH 769, 16 February 2015, VisIt 2.5.2; the same text in its 2017 version for IB2d), "Make a Movie of the Simulation": the wizard page by page, and "VisIt makes OK mpeg movies, but if you are readying this movie for a presentation, it will be better quality if you output TIFF or PNG images and stitch them together in a video player like Quicktime Pro".
- "3B – Analyzing your IBFE simulation" (2018), slides 3–8: record the GUI in Python, add a loop that saves a picture at every time, and run it with `visit -cli -nowin -s` on the cluster.

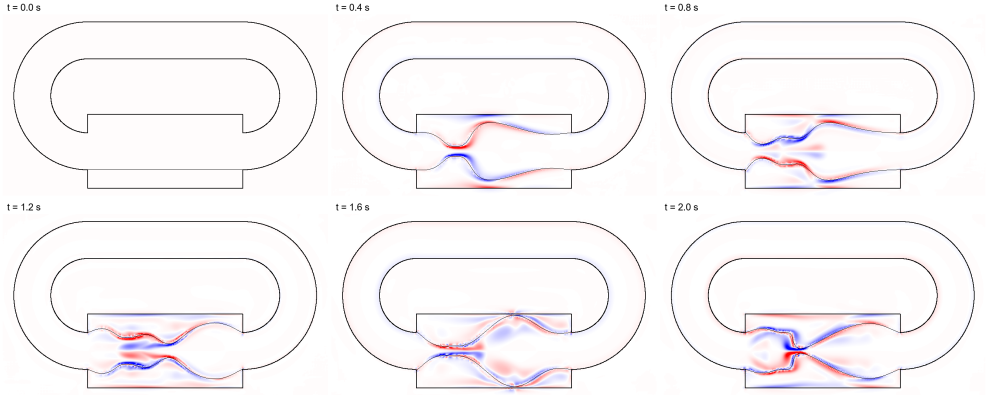
Chapter 4b already makes an IB2d movie this way (the wizard with PNG, then an ffmpeg line); this chapter is the whole story. New here: frames + ffmpeg and `visit -movie`, which no deck taught; `visit -movie` (the wizard's own engine) and a loop of your own run on IB/11 in VisIt 3.5.0 (Linux, no window) with ffmpeg 6.1.1, with what each writes, how long it lasts and how big it is; the wizard's 3.5 pages read from the program itself (the wizard itself was not run); and three scripts, `examples/analysis/visit_movie_frames.py`, `make_movie.py` and `movie_clean.py`.

WHAT THE OLD DECKS TAUGHT, AND WHAT CHANGED

then	now
File → Save movie → MPEG movie (2014–2024)	the wizard is still there in 3.5, and lists MP4 beside MPEG when it finds ffmpeg (VisIt's movie formats are made by ffmpeg); ask it for PNG images and encode them yourself (slide 6)
"Save as mpeg is not the best option" (2019–2021)	measured: VisIt's MPEG is MPEG-1 at 24 frames/s (it asks ffmpeg for 18 Mbit/s; ours averages 2.6), 2.0 MB for our 51 frames; ffmpeg's H.264 MP4 of the same frames is 0.56 MB (slide 14)
the wizard's frame rate	VisIt copies frames to make 30 frames/s, then ffmpeg plays the copies at 25 (MP4) or writes 24 (MPEG): 10 comes out as 8.3 (slide 15)
VirtualDub ("free for PC") or QuickTime Pro to join the images	ffmpeg: free, the same command on Linux, macOS, Windows and the cluster. QuickTime 7 Pro is a 32-bit program, which macOS 10.15 and later do not run (Apple ended its Windows version in 2016); VirtualDub is Windows-only, last released (1.10.4) in 2013
the 2018 loop: "Note we advance in time to skip t=0"	start at dump 0, and step every time slider (slide 24)
the coloured boxes hidden with **Hide**/**Show**	delete the Subset plot of the levels (slide 10)
Killdevil's VisIt 2.12.3 and Python 2	VisIt 3.5's Python 3; Puma in chapter 31

The decks of the previous slide; measurements: `wsl/final/ch30_movies*.sh`, `ch30_mp4.sh`.

THE EXAMPLE: IB/11'S RACETRACK, 51 DUMPS



IB/11 (chapter 9b) in mode `TARGET_VP`: a loop of tube 10 cm in diameter (the loop about 0.82 by 0.40 m), pumped by squeezing its elastic bottom section once a second; 2 s (two periods) at `DT` $= 5 \times 10^{-5}$ s with a dump every 800 steps (0.04 s): 51 dumps, so 51 frames. Shown: the vorticity `Omega` (colour table `difference`, fixed at ± 103.1 s^{-1}), the rigid racetrack (its vertices), the elastic tube (its spring mesh) and the rigid pericardium box. Above: dumps 0, 10, . . . , 50.

Frames by `wsl/final/ch30_frames.sh` (VisIt 3.5.0, 1200 $\times$ 800); montage `tools/ch30_figures.py`.

THREE ROUTES TO THE SAME FRAMES

route	how	what you get	use it for
1. the wizard	**File** → **Save movie**, six pages for a simple movie (slides 12–13)	a movie file, or the frames themselves if you choose an image format	one movie, now, from the GUI
2. `visit -movie`	a saved session (or a script) and one shell command (slides 16–23)	the same, without the GUI; `-source` swaps the data, but steps one time slider only (slide 21)	the same movie for several runs; a batch job
3. your own loop	`SaveWindow()` in a `for` loop, run by `visit -nowin -cli -s` (slides 24–25)	frames named as you like, and a table of their times	full control; the scripted analysis of chapters 27–29; the cluster

Routes 1 and 2 are the same program inside: the wizard writes a few lines of Python for the movie class of VisIt's `makemovie.py` (`movie.GenerateFrames()`, then `movie.EncodeFrames()`), which renders every frame and then calls ffmpeg through `visit_utils/encoding.py`.

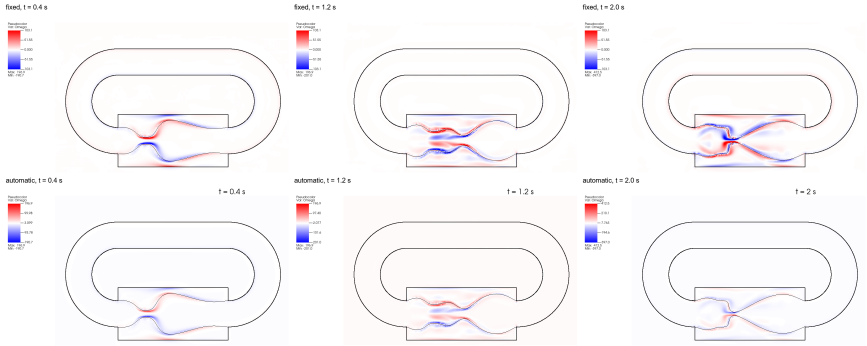
> Whatever the route, ask for PNG frames and run ffmpeg yourself (slide 26): you keep the frames, you set the frame rate, and a new encoding takes a second instead of a new render.

VisIt 3.5.0: `bin/makemovie.py`, `lib/site-packages/visit_utils/encoding.py`, and the strings of `lib/libgui.so`.

BEFORE THE FIRST FRAME

1. **Options** → **Preferences**: untick **Enable warning message popups**. Every old deck starts here: “when we start saving movies, a pop-up will spring up every time-step”.
2. Delete the Subset plot of `levels` that opening `dumps.visit` adds (slide 10).
3. Fix the colour limits (slide 8).
4. Fix the view: zoom once to the region that matters for the whole run (the structure moves). The view is saved with the session.
5. One legend: keep the colour bar of the field, untick **Legend** on the structure’s plots. Waldrop: “Most of the time, things like legends will need to be removed as to not clutter the plot.”
6. A time stamp (slide 9); no user name, no database path.
7. The size in pixels: even numbers (H.264 needs them; slide 27), e.g. 1200 × 800 or 1920 × 1080, set in the wizard (**Specify movie size**) or in the save options.
8. Waldrop: “Make sure that your Active Window is the window of the plot that you are trying to make the movie of.”
9. Play it once with the time slider (2014: “press the Play button and watch the simulation!”), then **File** → **Save session**: the session is the movie’s recipe, and route 2 replays it.

FIX THE COLOUR LIMITS



- Automatic limits follow each frame: -190.7 to 196.9 at 0.4 s, -201.0 to 196.9 at 1.2 s, -397.0 to 412.5 at 2.0 s. In a movie the scale breathes: the same red means twice the vorticity at the end, and a strong vortex can look weaker than a faint one. An uneven range also moves zero off the white middle of `difference`, so the still fluid turns blue (middles 3.1 and 7.8) or pink (-2.1).
- Fixed: “Try 25% of original min/max” (2020, slide 24): a quarter of the largest $|\Omega|$ at the last frame, $\pm 103.1\ \mathrm{s}^{-1}$ (**Limits**: tick **Minimum** and **Maximum**). The cores saturate: say so in the caption. The legend’s **Max** and **Min** lines still report each frame’s own extremes.

`wsl/final/ch30_autolim.sh` (automatic limits, MinMax per frame) and `ch30_frames.sh` (fixed).

THE TIME STAMP

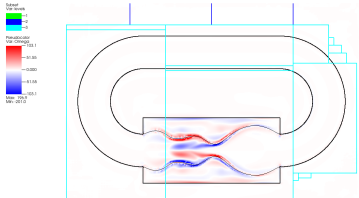


- In VisIt 3.5 the cycle and the time are printed only under the database's name ("DB: summary.samrai", then "Cycle: 32000 Time:1.6"): untick **Database** in **Controls** → **Annotation** → **General** and they go too, whatever **Time** says (top right).
- For the time alone: **Controls** → **Annotation** → **Objects**, a **Text 2D** object with the text `t = $time s`. VisIt replaces `$time` at every frame, in its shortest form: `t = 0.04 s`, `t = 1 s`, `t = 2 s` (the width changes, so put it where that does not matter). `$cycle` gives the step (800, 20000, 40000) and `$index` the dump (1, 25, 50, of `$numstates` = 51).
- In a script: `t = CreateAnnotationObject("Text2D"); t.text = "t = $time s"; t.position = (0.72, 0.92); t.height = 0.035`.
- The unit is yours to write: IBAMR's time is in the units of the input file (seconds for IB/11, which is in SI).

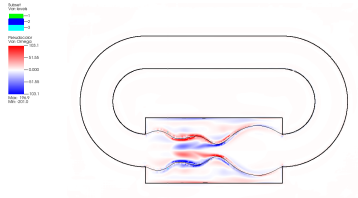
`wsl/final/ch30_timeannot.sh` (dump 40, $t = 1.6$ s) and `ch30_misc.sh` (dumps 1, 25, 50). Chapter 26 corrected to match.

THE COLOURED LINES: DELETE THE SUBSET PLOT

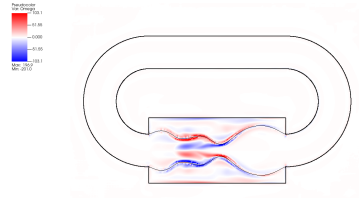
Subset plot of levels left in



the same, after movie_clean.py



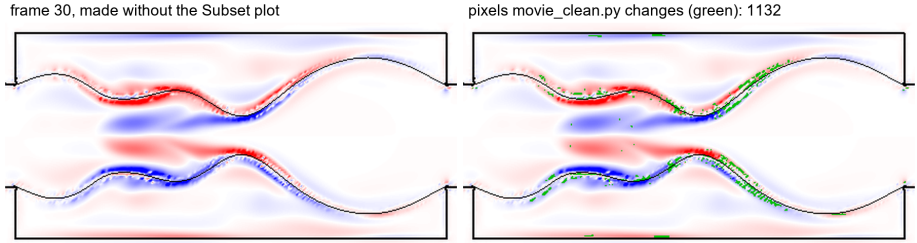
Subset plot deleted first



- Opening `dumps.visit` adds a Subset plot of the variable `levels` by itself: the plot list right after `OpenDatabase` holds it (chapter 26). Its lines are the patch boundaries of the refinement levels, and they move at every regrid: the “colored boxes” of the 2014 tutorial, which it hid with **Hide**/**Show**. Waldrop’s guide deletes it: “Most of the time, these just clutter your plot.
- For a movie, delete it: select it and **Delete** in the GUI; `DeleteAllPlots()` right after `OpenDatabase` in a script (all of this chapter’s scripts do).
- Frames already made: `examples/analysis/movie_clean.py` (next slide) took the lines out of the left frame (middle), but not the Subset legend.

`wsl/final/ch30_levels.sh`: dump 30 rendered with the Subset plot left in, then cleaned; right, the same dump from `ch30_frames.sh`.

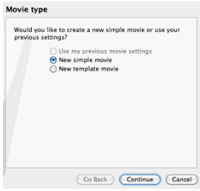
WHAT THE FRAME CLEANER COSTS

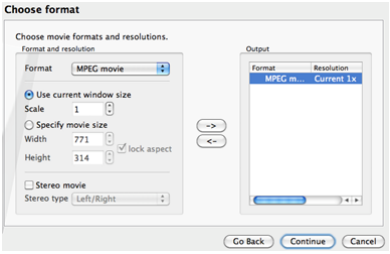


- `python movie_clean.py <frames folder> [glob, default f*.png] [K, default 1]` writes cleaned copies to `<folder>/clean/`. A line pixel is a saturated primary or secondary colour (red, green, blue, cyan, magenta, yellow) in a feature at most $2K$ pixels thick; it gets the mean colour of its neighbours. Green is always replaced (`difference` has none).
- The field's own thin saturated features look the same to it. On frames made without the Subset plot it still changed 691, 1132 and 1490 pixels (dumps 10, 30, 50): the saturated layers along the tube's walls, softened (above).
- So: a repair for old frames, not a step of the method. Delete the Subset plot and render again when you can.

`scratch/ch30/cleancheck` (the script run on three frames of `ch30_frames.sh`); `tools/ch30_figures2.py`.

ROUTE 1: FILE → SAVE MOVIE, PAGES 1 AND 2





- **File** → **Save movie** opens the **Save movie wizard**.
- **Movie type**: “Would you like to create a new simple movie or use your previous settings?” **New simple movie**, **Use my previous movie settings** (greyed out in the picture: there were no previous settings yet), or **New template movie** (several viewports and sequences, set up from a movie template).
- **Choose format**: a **Format**, then the size: **Use current window size** times a **Scale**, or **Specify movie size** with **lock aspect**; **Stereo movie** if you need it.
- Then the right arrow **->**: the format goes into the **Output** list. Waldrop: “Be sure to click the -> button to add your movie to the queue.” Several formats can be queued for one render, e.g. **PNG images** and **MP4 movie**.
- The formats VisIt 3.5 offers, and what they gave: slide 14.

VisIt 2.5.2 windows from N. Battista’s 2014 tutorial, reused in the 2020 deck (slides 21–22) and the 2024 practical (slides 25–26); the 3.5 page names and texts from `libgui.so`.

ROUTE 1: PAGES 3 TO 6

page	what it asks (VisIt 3.5)
Choose length	"Choose movie start/end time and frames per second": **Frames per second**, **First frame**, **Last frame**, **Frame stride** (2014: "if you want to make a movie out of every other frame, every 5, etc"), **Initial frame value** (the number of the first file)
Choose filename	"Choose the output directory and base filename for your movie(s)"; an image format adds the frame number: `racetrack0000.png`, ...
E-mail notification	"Do you want to be notified by E-mail when your movie completes?" (2014: "Typically, I say no")
Choose method	"Choose when and how you would like VisIt to create your movies": **Now, use currently allocated processors** (2014: "What is easiest (and gives the least probability that Visit will get buggy and close)"); **Now, use a new instance of VisIt** ("VisIt executed the following command line to begin making your movie"); **Later, tell me the command to run** ("Execute the following command at the command prompt ...": a `visit -movie` command for route 2)
not in a simple movie	**Save new template** ("Save this movie as a new template?"): only when you build a template movie; **Settings check** ("Do these movie settings look okay?"): only after **Use my previous movie settings**, where yes makes the movie and no goes back to **Choose format**

Then the frames are rendered one by one (Waldrop: on a Mac "this will open a terminal window and begin making the plots one at a time") and, for a movie format, encoded at the end. The frame rate you type here goes through the padding of slide 15.

Page names and quoted texts: VisIt 3.5.0's `libgui.so`; the order of the pages: `nextId()` in VisIt 3.5.0's `src/gui/QvisSaveMovieWizard.C`; the 2014 tutorial; Waldrop's guide.

THE FORMATS, AND WHAT THEY GAVE ON 51 FRAMES

VisIt 3.5's wizard lists **PNG**, **JPEG**, **TIFF**, **BMP**, **PPM** and **RGB images**, and **MP4**, **MPEG**, **AVI**, **DIVX**, **GIF**, **QuickTime**, **SWF** and **WMV movie**, all the movies but MPEG only when it finds `ffmpeg` on the PATH. The movies are made by ffmpeg: `encoding.py` offers mp4, mpg, wmv, gif, avi, mov, swf and divx only when `ffmpeg` is on the PATH, and MPEG falls back to VisIt's own `mpeg2encode` without it. VisIt renders JPEG frames for MP4, AVI, DIVX, GIF, SWF and WMV, and PNG frames for MPEG and QuickTime.

how (1200 × 800)	codec	frames/s	length	size
ffmpeg on the PNG frames, `-framerate 10`	H.264 High, yuv420p	10	5.10 s	0.56 MB
`visit -movie -format mp4` (`-fps 10`, the default)	H.264 High, yuv420p, from JPEG frames	25, each frame 3 times	6.12 s	0.58 MB
`visit -movie -format mp4 -fps 30`	the same	25	2.04 s	0.38 MB
`visit -movie -format mpeg -fps 10`	MPEG-1, 2.6 Mbit/s (18 asked for)	24	6.15 s	2.0 MB
ffmpeg, a GIF 600 pixels wide	256 colours	10	5.10 s	1.2 MB
ffmpeg without `-pix_fmt yuv420p`	H.264 High 4:4:4 Predictive, yuv444p: QuickTime and most players refuse it	10	5.10 s	0.56 MB

The PNG frames themselves: 4.0 MB for 51 (about 78 kB each). Rendering took 21 s; VisIt's MP4s 22–28 s with the encoding.

`wsl/final/ch30_movies.sh`, `ch30_movies4.sh`, `ch30_mp4.sh`, `ch30_examples.sh`; `ffprobe` for every number (the MPEG's bit rate: `scratch/ch30/verify_agent/va_run1.sh`).

THE FRAME-RATE TRAP

`makemovie.py`: “We can only drive the movie at 30 fps. So if the user wants a movie at 10 fps, we have to pad frames.”

- The padding: the smallest whole number i with $30/i \leq$ fps; each frame is copied i times (3 for 10 frames/s, 1 for 30). A rate that is not 30 divided by a whole number prints “Because of limitations in MPEG encoding, the movie will be encoded at . . . frames per second”.
- Then ffmpeg reads the copies at its default input rate for images, 25 per second (MP4), or writes MPEG-1 at 24 per second. Neither is 30.
- So `-fps 10` (and the wizard’s **Frames per second** 10) plays $25/3 = 8.3$ of your frames per second: 51 frames last 6.12 s, not 5.1. `-fps 30` plays 25 per second: 2.04 s.

> If the rate matters (a period that should last a known time), make PNG frames and give ffmpeg the rate yourself: `-framerate 10` gives exactly 10 per second, 5.10 s.

VisIt 3.5.0 `makemovie.py` (`EncodeMovie`, `EncodeMPEGMovie`) and `encoding.py` (`encode_mp4`, `encode_mpeg1`); measured with `wsl/final/ch30_mp4.sh`, `ch30_movies4.sh`.

ROUTE 2: VISIT -MOVIE FROM A SAVED SESSION

Set the plots up once (the GUI, or a script ending in `SaveSession("racetrack.session")`), save the session, then from a shell:

```
mkdir -p frames movies                                  # the folders must exist (slide 18)
visit -movie -format png -geometry 1200x800 \
      -sessionfile racetrack.session -output frames/f
# -> frames/f0000.png ... frames/f0050.png, one per dump
visit -movie -format png,mp4 -geometry 1200x800 -sessionfile racetrack.session -output movies/
  racetrack
```

- `visit -help`: “Run the CLI in a movie making mode. Must be combined with -sessionfile. Will produce a simple movie by drawing all the plots in the specified session for every timestep of the database.” (`-scriptfile` works too: slide 23.)
- In our runs (Linux, no display) adding `-nowin` changed nothing. On a cluster this is the command for a batch job (chapter 31); the wizard’s **Later, tell me the command to run** writes one for you.
- The session holds the plots, their attributes, the view, the annotations and the data files (`SOURCE00`, `SOURCE01`, ... in the XML); `-source` replaces the data (slide 20).

wsl/final/ch30_movies3.sh, ch30_outdir.sh; the help text of `visit` 3.5.0.

VISIT -MOVIE: THE OPTIONS

option	what it does (from `makemovie.py`'s usage, VisIt 3.5)
`-format f[,f2...]`	`png`, `jpeg`, `tiff`, `bmp`, `ppm`, `rgb` images; `mpeg`; with ffmpeg also `qt`, `mp4`, `wmv`, `gif`, `avi`, `swf`, `divx`; several at once with commas
`-geometry WxH[,W2xH2]`	the size; without it, the window size stored in the session (512 $\times$ 512 without a session)
`-sessionfile name`	the session to replay
`-source file` (repeat)	new data in place of the session's sources, in their order (only the active time slider is then stepped: slide 21); `-adjustview` resets the view to the new data's bounds, keeping the view normal and up vectors
`-scriptfile name`, `-templatefile name`	a Python script (slide 23) or a movie template instead of a session
`-start`, `-end`, `-framestep`	which time states (the files are numbered from 0 unless `-frame` is given)
`-frame value`	the number of the first file
`-output name`	folder and base name of the frames or the movie
`-fps number`	the frame rate (default 10; slide 15)
`-stereo f`	`off`, `leftright`, `redblue`, `redgreen`
`-ignoresessionengines`	do not restart the compute engines stored in the session
`-email a,b`	progress reports by e-mail
`-noffmpeg`	MPEG through VisIt's own `mpeg2encode`
`-enginerestartinterval n`	restart the compute engine every n images (memory, chapter 31)
`-np`, `-nn`, `-l`, `-la`, `-p`	a parallel engine: processors, nodes, the launcher (`mpirun`, `poe`, `psub`, `srun`), its arguments, the partition

`makemovie.py PrintUsage` (VisIt 3.5.0): `wsl/final/ch30_mmhelp.sh`, `ch30_mmhelp2.sh`; `-fps` default from its source.

WHAT `visit -movie` DOES NOT TELL YOU (1/2)

what we did	what it printed	what it wrote
`-output` into a folder that did not exist	"Saved . . . /vm/frame0000.png" for every frame, then "VisIt completed generating frames"	nothing, anywhere: create the folder first
`-sessionfile` naming a file that was not there	"No image was saved. The combination of pixel data flags resulted in no image." . . . "VisIt completed generating frames"	nothing
`-scriptfile` with a script that only made the plots	"VisIt completed generating frames"	nothing (slide 23)
`-start 10 -end 20 -framestep 5`		`frame0000.png`–`frame0002.png` (dumps 10, 15, 20)

`wsl/final/ch30_movies.sh`, `ch30_movies2.sh`, `ch30_movies4.sh`, `ch30_outdir.sh` (VisIt 3.5.0).

WHAT `VISIT -MOVIE` DOES NOT TELL YOU (2/2)

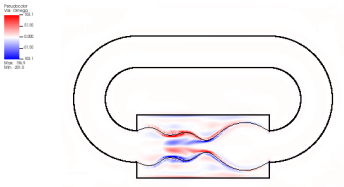
what we did	what it printed	what it wrote
`-format png,mp4 -output movies/racetrack`, with `movies/` missing	a traceback ending “FileNotFoundError: [Errno 2] No such file or directory: ’. . . /movies”’	nothing: with a movie format VisIt lists the folder first, and stops
`-source` for another run with a structure	nothing unusual	frames whose structure stays at dump 0 (slide 21)
`-start 10 -end 20 -framestep 5 -frame 10 -output .../g`		`g0010.png`–`g0012.png`
the runs from a session	“ERROR: We could not parse the session file for engine launch parameters . . . Please report this.”	the frames, correct: the message is harmless here
any run		`<base>.log` in the working folder (`frame.log` for `-output .../frame`), and `visitlog.py`, as every CLI run

Count the files after every run (`ls frames | wc -l`): “completed” does not mean “written”.

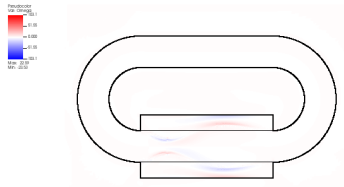
`wsl/final/ch30_outdir.sh`, `ch30_source.sh`; `scratch/ch30/verify_agent/va_run2.sh` (D1) (VisIt 3.5.0).

ONE SESSION, ANOTHER RUN: -source

TARGET_VP (the session as saved)



FORCE_VP by -source: the tube as at t = 0



```
visit -movie -format png -geometry 600x400 -sessionfile racetrack.session \
      -source FORCE_VP_F025/viz_IB2d/lag_data.visit -source FORCE_VP_F025/viz_IB2d/dumps.visit \
      -output force/frame                    # the session's SOURCE00 is lag_data.visit, SOURCE01
  dumps.visit
ffmpeg -framerate 10 -i target/frame%04d.png -framerate 10 -i force/frame%04d.png \
       -filter_complex "[0:v][1:v]hstack=inputs=2" -c:v libx264 -pix_fmt yuv420p -crf 18 both.mp4
```

51 frames of the force-driven pump in 14 s, with the session's plots, view and limits; then side by side, 1200 $\times$ 400, 5.1 s, 0.25 MB. Same limits on both sides is the point: at this dump $|\Omega|$ reaches 23.5 s^{-1} in the force-driven pump and 201 in the target-driven one. But the right-hand tube is straight, as at $t = 0$, in all 51 frames: `-source` moved the fluid's time slider only (next slide).

wsl/final/ch30_source.sh; IB/11's FORCE_VP run with Fmag_vp $= 0.25$ (chapter 9b).

-source: only one time slider moves

- With `-source`, `makemovie.py` restores the session with the new files, re-opens each of them ("Restoring the session is messing up adding the new time sliders from the new databases to the new window"), then steps the *active* time slider only (`IterateAndSaveFrames`: `SetTimeSliderState(i)`).
- The active slider is then the last file re-opened, the new `dumps.visit`: `SetTimeSliderState(25)` left `lag_data.visit` and `Correlation01` at 0. In all 51 frames of the previous slide the force-driven tube's walls sit on the same pixel rows (239 and 289 of 400), as at $t = 0$; drawn at dump 30 they are squeezed.
- Without `-source` the replay is right: the `TARGET_VP` frames are pixel for pixel those of the script (dumps 10, 15, 20 compared).
- For another run with a structure, step every slider: your own loop (slide 24), or `visit_movie_frames.py` with the session's limits, structures, view and size:

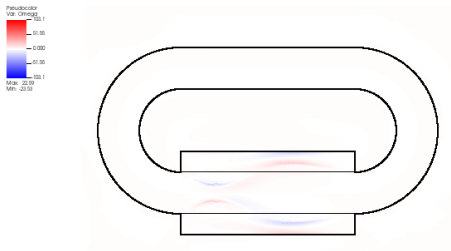
```
visit -nowin -cli -s visit_movie_frames.py -- FORCE_VP_F025/viz_IB2d force lim=103.1 \
      meshes=heart_race_512_vertices,heart_tube_512_mesh,heart_peri_512_vertices \
      window=-0.45,0.45,-0.3,0.3 size=600x400 time=0 prefix=frame
```

51 frames in 19.3 s, `force/frame0000.png` … `frame0050.png`; with the `TARGET_VP` frames and the `hstack` command of the previous slide: 1200 × 400, 5.1 s, 0.26 MB, both tubes squeezed at dump 30.

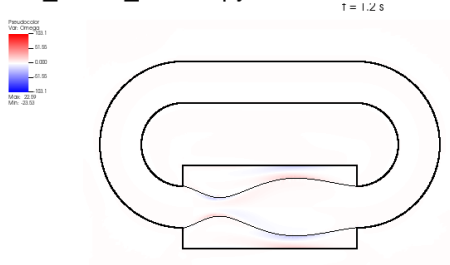
VisIt 3.5.0 `makemovie.py` (`GenerateFrames`, `IterateAndSaveFrames`); `scratch/ch30/verify_agent/va_run6.sh`–`va_run8.sh`; the walls' rows: `checks.py` there, on the frames of `ch30_source.sh`.

THE SAME DUMP, TWO WAYS

visit -movie -source: the tube as at t = 0



visit_movie_frames.py: the tube at t = 1.2 s



- `FORCE_VP`, dump 30 ($t = 1.2$ s), the same limits and view. Left, from `visit -movie -source`: the fluid of dump 30 around the walls of dump 0. Right, from `visit_movie_frames.py`: the walls of dump 30, squeezed by the applied force.
- Nothing in the left frame looks wrong: the vorticity is right, the structure is plausible, and VisIt reported no problem. Only a frame from another route shows it. Check one frame of every new route against a frame you trust.

`scratch/ch30/verify_agent` (`va_run7.sh`, `va_run8.sh`); `tools/ch30_figures2.py`.

VISIT -MOVIE -SCRIPTFILE: YOUR SCRIPT STEPS THROUGH TIME

With `-scriptfile` VisIt runs your script once, after replacing every `SaveWindow()` in it by its own numbered save (`CreateMangledSource` in `makemovie.py`). It does not step through time for you.

```
# ... OpenDatabase, DeleteAllPlots, AddPlot, attributes, DrawPlots, the view, the Text2D (as in the loop of slide 24)
for k in range(0, 51, 10):
    for ts in GetTimeSliders():
        SetActiveTimeSlider(ts); SetTimeSliderState(k)
    SaveWindow()
```

- `visit -movie -nowin -format png -geometry 900x600 -scriptfile movie.py -output sf/frame`: 6 files, `frame0000.png`–`frame0005.png`, for dumps 0, 10, . . . , 50. The same script without the loop and `SaveWindow()`: no files.
- The rewrite keeps one character after `SaveWindow()` and drops the rest of the line, newline included. `SaveWindow(); n += 1` lost `n += 1` and glued the next line, `print(...)`, onto the end of the loop's last line: it printed six times, with `n` still 0.
- So write `SaveWindow()` exactly so, alone at the end of its line, and end the file with a newline: a script whose last line is `SaveWindow()` with no newline stopped with "IndexError: string index out of range" and wrote nothing.

wsl/final/ch30_scriptfile.sh, ch30_scriptfile2.sh; makemovie.py 3.5.0, GenerateFrames and CreateMangledSource.

ROUTE 3: YOUR OWN LOOP

```
import os, sys; R = "viz_IB2d/"; os.makedirs("frames", exist_ok=True)
OpenDatabase(R + "lag_data.visit"); DeleteAllPlots()
OpenDatabase(R + "dumps.visit"); DeleteAllPlots()   # the levels plot
a = AnnotationAttributes()
a.userInfoFlag = 0; a.databaseInfoFlag = 0; a.axes2D.visible = 0
SetAnnotationAttributes(a)
t = CreateAnnotationObject("Text2D"); t.text = "t = $time s"
t.position = (0.72, 0.92); t.height = 0.035
AddPlot("Pseudocolor", "Omega"); p = PseudocolorAttributes()
p.colorTableName = "difference"; p.centering = p.Nodal
p.minFlag = 1; p.min = -103.1; p.maxFlag = 1; p.max = 103.1
SetPlotOptions(p); m = MeshAttributes(); m.legendFlag = 0
ActivateDatabase(R + "lag_data.visit")
AddPlot("Mesh", "heart_tube_512_mesh"); SetPlotOptions(m); DrawPlots()
v = GetView2D(); v.windowCoords = (-0.45, 0.45, -0.3, 0.3); SetView2D(v)
s = SaveWindowAttributes()
s.outputToCurrentDirectory = 0; s.outputDirectory = "frames"
s.family = 0; s.format = s.PNG
s.width, s.height = 1200, 800; s.resConstraint = s.NoConstraint
for k in range(TimeSliderGetNStates()):
    for ts in GetTimeSliders():       # fluid, structure, correlation
        SetActiveTimeSlider(ts); SetTimeSliderState(k)
    s.fileName = "f%04d" % k
    SetSaveWindowAttributes(s)
    SaveWindow()
sys.exit()
```

- Run it with `visit -nowin -cli -s frames.py`. The `sys.exit()` at the end matters: without it the CLI waits at its prompt. `SaveWindow()` makes no folder: hence `os.makedirs`.
- Every time slider: with both files open, VisIt has one slider per database and a correlation of the two. Setting the active one alone left the others where they were (dumps.visit at 50, lag_data.visit and the correlation at 0): a moving fluid around a frozen structure.
- From dump 0: the 2018 loop called `TimeSliderNextState()` before its first save ("to skip t=0"), so the initial state was never saved; it named the files with `family = 1` (VisIt numbers them: `visit0000.png`, ...) and kept the window's proportions (`ScreenProportions`).
- Naming the files yourself ties frame k to dump k; `NoConstraint` gives exactly the size asked for.

The 2018 deck "3B", slide 7, rewritten for VisIt 3.5 (Python 3); time sliders: `wsl/final/ch30_frames.sh`; the listing: `verify_agent/va_run1.sh`, `va_run9.sh`.

THE LOOP AS A SCRIPT: `visit_movie_frames.py`

```
visit -nowin -cli -s examples/analysis/visit_movie_frames.py -- <viz folder or dumps.visit> <out dir> [key=value ...]
    var=Omega  lim=auto  ct=difference  meshes=all  window=xmin,xmax,ymin,ymax  size=1200x800
    start=0 end=-1 every=1  first=0  time=1  legend=1  prefix=f  session=<file>
```

- `lim=auto`: a quarter of the largest |value| at the last frame, the same for every frame (on IB/11: "Omega at the last frame: min -396.983, max 412.509; limits -103.127 .. 103.127"); `lim=L` for $\pm L$; `lim=lo,hi`.
- `var=` also takes an expression: `"var=magnitude(U)" lim=0,0.5 ct=hot` for the speed (IBAMR 0.19 writes the vector `U` only).
- `meshes=all`: every `<name>_mesh` of `lag_data.visit`, and `<name>_vertices` for structures without one (on IB/11 that adds the actuators and the markers); or a list: `meshes=heart_race_512_vertices,heart_tube_512_mesh,heart_peri_512_vertices`; `meshes=none`.
- It deletes the Subset plot, writes `f0000.png`, ... numbered without gaps whatever `every` is, and `frames.csv` (frame, dump, t); `first=` numbers a later chunk on (`start=25 end=50 first=25`: `f0025.png` ..., `frames.csv` appended); `session=` also saves the set-up for `visit -movie`.
- On IB/11: 51 frames in 21.3 s; every fifth dump, 11 frames in 4.7 s.

`examples/analysis/visit_movie_frames.py`, tested by `wsl/final/ch30_examples.sh` (VisIt 3.5.0).

FFMPEG: ONE COMMAND, FLAG BY FLAG

```
ffmpeg -framerate 10 -i frames/f%04d.png -c:v libx264 -pix_fmt yuv420p -crf 18 -movflags +faststart
   racetrack.mp4
```

`-framerate 10`	before `-i`: how many frames go by per second (`-r` after `-i` changes the rate of the output by copying or dropping frames: slide 29)
`-i frames/f%04d.png`	`f0000.png`, `f0001.png`, ..., consecutive from 0 (or from `-start_number N`)
`-c:v libx264`	H.264 video
`-pix_fmt yuv420p`	colour at half resolution, the form players expect (slide 27)
`-crf 18`	the quality: 0 is lossless, 23 ffmpeg's default, 51 the worst; "17 or 18" is "visually lossless or nearly so" (FFmpeg's H.264 guide)
`-movflags +faststart`	the index at the front of the file, so a browser starts playing before the download ends

On IB/11: 51 frames of $1200 \times 800 \rightarrow 5.10$ s, 0.56 MB, in under a second. Getting ffmpeg: `sudo apt install ffmpeg` (Ubuntu, and WSL: 6.1.1 on Ubuntu 24.04), `brew install ffmpeg` (macOS), a Windows build linked from https://ffmpeg.org/download.html (add its `bin` folder to the PATH); on Puma, look with `module avail` (chapter 31).

`wsl/final/ch30_movies.sh`; FFmpeg's H.264 encoding guide, https://trac.ffmpeg.org/wiki/Encode/H.264.

FFMPEG: THE TRAPS (1/2)

what we did	what happened	what to do
frames of 1199 × 799	"width not divisible by 2 (1199x799)", "Error while opening encoder", and an empty (0-byte) file	`-vf "pad=ceil(iw/2)*2:ceil(ih/2)*2"` (adds a pixel; VisIt's own MP4 encoder does this) or `scale=trunc(iw/2)*2:trunc(ih/2)*2` (1198 × 798)
no `-pix_fmt` on PNG frames	H.264 "High 4:4:4 Predictive", yuv444p: full colour resolution, which "QuickTime and most other players" do not play (FFmpeg's guide)	always `-pix_fmt yuv420p`
frames 0, 1, 3 (2 missing)	`-i f%04d.png` stopped at the gap: 2 frames	renumber, or `-pattern_type glob -i "f*.png"` (3 frames; needs an ffmpeg built with globbing, which the usual Windows builds are not), or `make_movie.py`

ffmpeg 6.1.1 on the IB/11 frames: `wsl/final/ch30_movies.sh`; https://trac.ffmpeg.org/wiki/Encode/H.264.

FFMPEG: THE TRAPS (2/2)

what we did	what happened	what to do
frames that start at 10	"Could find no file with path 'f%04d.png' and index in the range 0-4", and no file	`-start_number 10` (41 frames from `f0010.png`)
`-framerate 10` after `-i`	no error; the images went by at ffmpeg's default for images, 25 per second: 51 frames in 2.04 s	`-framerate` before `-i`
UArizona's VisIt page: `ffmpeg -f image2 -i movie%04d.png -r 10 output.mp4`	the images read at 25 per second, then thinned to 10: 22 of the 51 frames kept, 2.2 s; and yuv444p	`-framerate 10` before `-i`, and `-pix_fmt yuv420p`: 51 frames, 5.1 s

ffmpeg 6.1.1 on the IB/11 frames: `wsl/final/ch30_movies.sh`, `ch30_misc.sh`, `ch30_uahpc_ffmpeg.sh`; https://trac.ffmpeg.org/wiki/Encode/H.264; UArizona HPC documentation, VisIt page (read 17 October 2026).

MORE WITH FFMPEG (1/2)

```
# a GIF with its own 256-colour palette, 600 pixels wide          -> 1.2 MB (the full-size MP4: 0.56 MB)
ffmpeg -framerate 10 -i f%04d.png -vf "scale=600:-1:flags=lanczos,split[a][b];[a]palettegen[p];[b][p]paletteuse" race.gif
# two runs side by side (slide 20); vstack=inputs=2 puts them one above the other
ffmpeg -framerate 10 -i a/f%04d.png -framerate 10 -i b/f%04d.png -filter_complex "[0:v][1:v]hstack=inputs=2" \
       -c:v libx264 -pix_fmt yuv420p both.mp4
# twice as slow: every frame shown twice as long                  -> 10.1 s instead of 5.1
ffmpeg -framerate 10 -i f%04d.png -vf "setpts=2*PTS" -c:v libx264 -pix_fmt yuv420p slow.mp4
# the same 51 frames at 30 frames/s: each repeated three times    -> 153 frames, still 5.1 s, no smoother
ffmpeg -framerate 10 -i f%04d.png -r 30 -c:v libx264 -pix_fmt yuv420p r30.mp4
```

Smoothness comes from more frames (more dumps), never from `-r` or `setpts`: slide 32.

All run on the IB/11 frames with ffmpeg 6.1.1: `wsl/final/ch30_movies.sh`, `ch30_source.sh`, `ch30_misc.sh`.

MORE WITH FFMPEG (2/2)

```
# a region only: width:height:x:y in pixels (the elastic section)  -> 440 x 220
ffmpeg -framerate 10 -i f%04d.png -vf "crop=440:220:470:420" -c:v libx264 -pix_fmt yuv420p tube.mp4
# one frame back out of a movie (frame 30, counted from 0)
ffmpeg -i tube.mp4 -vf "select=eq(n\,30)" -vframes 1 tube_030.png
# a title card shown for 2 s, then the movie (title.png the same size as the frames)  -> 71 frames, 7.1 s
ffmpeg -loop 1 -framerate 10 -t 2 -i title.png -framerate 10 -i f%04d.png \
       -filter_complex "[0:v][1:v]concat=n=2:v=1:a=0,format=yuv420p" -c:v libx264 -crf 18 titled.mp4
```

- The title card is any picture of the frames' size (a slide exported as PNG, or made in Python): what is shown, the run, the units, the slowdown and the colour limits, for the viewer who sees the movie without your talk. `-loop 1 -t 2` turns the still into 2 s of video at the movie's rate; `concat` joins the two.
- Chapter 4b promised the title frame here: it is the "what am I looking at" that a movie on a web page or in a supplement does not otherwise have.

ffmpeg 6.1.1 on the IB/11 frames: `wsl/final/ch30_misc.sh`, `ch30_title.sh`; the crop: `scratch/ch30/verify_agent/va_run3.sh`.

MAKE_MOVIE.PY: FRAMES TO A MOVIE ON ANY SYSTEM

```
python make_movie.py frames                                    # frames/*.png -> frames/movie.mp4 at
  10 frames/s
python make_movie.py frames --glob "f*.png" --fps 25 --crf 18 --out racetrack.mp4
python make_movie.py frames --gif --width 600 --out racetrack.gif
```

- It copies the frames, in name order, to a temporary folder as `00000.png`, `00001.png`, ... (so gaps and odd first numbers do not matter, and no glob is needed), pads the size to even numbers, sets `-pix_fmt yuv420p`, prints the ffmpeg command it runs and the movie's length and size.
- Tested (ffmpeg 6.1.1) on the frames of `visit_movie_frames.py`: 51 frames → 5.10 s, 571 kB, H.264 High, yuv420p, 10 frames/s; every other frame of the first 50 (25 files) → 25 frames; frames of 601 × 401 → a movie of 602 × 402; `--gif --width 600` → 1208 kB.
- It needs Python 3 and ffmpeg on the PATH, nothing else; it says where to get ffmpeg if it is missing.

`examples/analysis/make_movie.py`, tested by `wsl/final/ch30_examples.sh` (Ubuntu 24.04 in WSL).

CHOOSING THE FRAME RATE AND THE DUMP INTERVAL

- One frame per dump. IB/11's 51 dumps at 10 frames/s last 5.1 s: each 0.04 s of the run is shown for 0.1 s, so the run is slowed 2.5 times, one pumping period in 2.5 s. At 25 frames/s (real time) the whole run is over in 2.04 s.
- Rates of 10 to 30 frames/s are usual; at the low end the motion is jerky. Slowing a few frames down (`setpts`, `-r`) does not add motion between them.
- Plan backwards from the movie: length $\times$ rate = frames; the run's time / frames = the dump interval. A 10 s movie of IB/11 at 25 frames/s needs 250 frames over 2 s: a dump every 0.008 s, `VIZ_INTERVAL = 160` steps of 5×10^{-5} s in its input (now `int(END_TIME/(50*DT))` $= 800$).
- The disk: IB/11's dumps are about 2.5 MB each, 2.6 MB with the structure's files (133 MB for 51); 250 would be about 650 MB. A 3D run's dumps are far bigger: dump often only over the interval the movie shows (restart from a restart file with a smaller interval in the input, chapter 3; IB/11 writes none, `restart_dump_interval = 0`, so set it before the run; after a restart `dumps.visit` lists only the later dumps, chapter 33), or make the frames as the run goes and keep fewer dumps.
- Write the slowdown in the caption: "2 s of simulated time shown in 5.1 s".

IB/11's input (`DT`, `END_TIME`, `VIZ_INTERVAL`) and `du -sb` of its `viz_IB2d` folder (`scratch/ch30/verify_agent/va_run1.sh`).

MOVIES IN TALKS, PAPERS AND ON THE WEB

- Slides: H.264 in an MP4 with `yuv420p` is the format PowerPoint, Keynote and web browsers all play. Test it on the computer you will present from, and put a still of the key frame on a backup slide.
- Beamer and PDF: most PDF viewers do not play embedded video; link the file and play it in a player, or show the still with the link.
- Papers: supplementary movies as MP4; read the journal's limits on size and format first. Keep the frames: a figure panel is a frame you already have.
- The web and social media: MP4 with `+faststart`; a GIF only for a short loop (ours is twice the size of the MP4 at half the width).
- The caption: the variable and its colour limits ("saturated at ± 103 s^{-1}"), the time stamp's units, the slowdown, the run (the data sheet of chapter 29).
- The record: keep the session or the script, the frames (or the command that remakes them) and the encoding command beside the run. Waldrop's rule holds for movies too: someone else should be able to remake it without asking you.

MOVIES ON THE CLUSTER

- Every route but the wizard runs without a display: `visit -nowin -cli -s visit_movie_frames.py --...` or `visit -movie ...` inside a batch job, next to the data (the dumps of a 3D run are too big to copy home just for a movie).
- `visit -movie` can launch a parallel compute engine itself: `-np`, `-nn`, `-l srun`, `-la`, `-p` (slide 17).
- Long movies: `-enginerestartinterval n` restarts the compute engine every n images (for a render that slows down or runs out of memory), and a script can do its frames in chunks, each with its own `start=`, `end=` and `first=`.
- Encode on the cluster if it has ffmpeg, or copy the frames home (4.0 MB for our 51) and encode there.
- Puma's VisIt module, client–server use and a Slurm script for frames: chapter 31.

`makemovie.py`'s usage (VisIt 3.5.0); the 2018 deck "3B", slide 8 (a VisIt script run on Killdevil).

WHEN THE MOVIE GOES WRONG

what you see	what to do
a pop-up at every frame	untick **Enable warning message popups** (slide 7)
coloured straight lines that jump around	delete the Subset plot of `levels` (slide 10)
the colours flicker, or the still fluid changes colour	fixed limits (slide 8)
no time on the frames	a **Text 2D** with `$time` (slide 9)
“completed” or “Saved”, but no files	create the output folder: VisIt does not, in `visit -movie` or in your own loop (ours printed “Saved” 51 times and wrote nothing); check the session’s path; a `-scriptfile` must call `SaveWindow()` (slide 18)
the fluid moves, the structure does not	set every time slider (slide 24); after `visit -movie -source`, make the frames with your own loop (slide 21)
the movie plays slower (and lasts longer) than asked	VisIt’s padding (slide 15): give the rate to ffmpeg
“width not divisible by 2”	pad to even sizes (slide 27)
the MP4 will not play in QuickTime or another player	`-pix_fmt yuv420p`
ffmpeg makes a movie of only the first few frames	a gap in the numbering: renumber or `make_movie.py`
a jerky movie	more dumps, not a lower rate (slide 32)
the legend’s numbers change from frame to frame	they are the frame’s **Max** and **Min**: the limits are on the bar’s ends

The 2014–2024 decks and what happened while this chapter was made (VisIt 3.5.0, ffmpeg 6.1.1).

EXERCISES

1. Make the IB/11 vorticity movie three ways (the wizard with PNG images, `visit -movie` from your saved session, `visit_movie_frames.py`), encode each with ffmpeg at 10 frames/s, and compare the frames and the files.
2. Ask the wizard (or `visit -movie`) for an MP4 at 10 frames/s and measure its length with `ffprobe`. Explain the number.
3. Make a movie with automatic limits and one with fixed limits; watch both. Which vortex looks strongest in each, at 1.2 s and at 2.0 s?
4. Replay your session on IB/11's `FORCE_VP` run with `-source` and put the two movies side by side. What do the same limits show that separate limits would hide? Does the tube move? Make the `FORCE_VP` frames again with `visit_movie_frames.py` (same limits and view) and compare.
5. Make a movie of the speed, `magnitude(U)`, with the `hot` colour table, fixed limits 0 to 0.5 m/s and the markers drawn as points.
6. Plan a 10 s movie of one pumping period at 25 frames/s: the dump interval, the number of dumps, the disk. Rerun IB/11 for one period with that interval and make it.
7. Crop the movie to the elastic section and make a GIF of it under 1 MB.
8. Write the caption and the record (session, commands, data sheet) for one of your movies, so that someone else could remake it.

SUMMARY

- A movie is frames plus an encoder. Make PNG frames (the wizard with an image format, `visit -movie -format png`, or your own loop), keep them, and encode with ffmpeg: `-framerate`, `libx264`, `yuv420p`, `-crf 18`, `+faststart`.
- Before the first frame: no pop-ups, no Subset plot, fixed limits, a fixed view, one legend, a `$time` stamp, an even size.
- VisIt's own movie formats pad frames for 30 frames/s and are read back at 25 per second (MPEG is written at 24): the rate you get is not the rate you typed. Its MPEG is MPEG-1 and 3.6 times the size of an H.264 MP4.
- `visit -movie` replays a session (`-start`, `-end`, `-framestep`, `-frame`; `-source` for another run, but it steps one time slider only: our structure stayed at dump 0); it says "completed" when it wrote nothing, so count the files. With `-scriptfile` your script loops and calls `SaveWindow()` on its own line.
- Choose the movie first (its length and rate), then the dump interval, and write the slowdown and the limits in the caption.

> The old advice holds: "Save as mpeg is not the best option." Save the frames, and let ffmpeg make the movie.

REFERENCES AND CREDITS

- N. Battista, “Part III: Let’s make groovy movies! – Visualizing the Data and Simulation in ViSiT”, in the Kure/Killdevil tutorial (2014) and its 2018 MATH 769 version; its VisIt 2.5.2 wizard windows are reused on slide 12.
- L. A. Miller, “VISIT – How to visualize IBAMR Data” (2020; 2021 version), “Lecture 9: Sampling, averaging, and analyzing data in Visit” (2019), “IB2D Practical” (2024), “3B – Analyzing your IBFE simulation” (2018).
- L. Waldrop, “Data analysis in VisIt”, MATH 769 (16 February 2015; VisIt 2.5.2, IBAMR data), “Make a Movie of the Simulation”; the same section in its 2017 version for IB2d’s `.vtk` output.
- VisIt 3.5.0, Lawrence Livermore National Laboratory: https://visit-dav.github.io/visit-website/; its `makemovie.py` and `visit_utils/encoding.py`, read for slides 6, 15, 17, 21 and 23, and the wizard’s `src/gui/QvisSaveMovieWizard.C` for slide 13. H. Childs et al., “VisIt: An end-user tool for visualizing and analyzing very large data”, in *High Performance Visualization*, CRC Press (2012) 357–372.
- FFmpeg 6.1.1, https://ffmpeg.org; the H.264 encoding guide, https://trac.ffmpeg.org/wiki/Encode/H.264.
- The runs: `examples/IB/11-heart-tube-pump` (`TARGET_VP`; `FORCE_VP` with `Fmag_vp` $= 0.25$), IBAMR 0.19.0, chapter 9b.
- This chapter’s scripts: `examples/analysis/visit_movie_frames.py`, `make_movie.py`, `movie_clean.py`; in the working folder `wsl/final/ch30_*.sh`, `tools/ch30_figures.py`, `ch30_figures2.py`.